HW3 Problem 2 — Neural Bayes Estimators (torch version)

Team Members

2026-04-13

Load test data + Part 1 (MLE)

```
load("test_data.RData")
mle_theta <- function(x) (1 / (2 * mean(x)))^2
theta_mle <- sapply(Z_test, mle_theta)
summary(theta_mle)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.000000 0.004619 0.016170 0.351503 0.443490 3.222384
```

Part 2 — Neural Bayes Estimator (torch)

Setup

```
library(torch)
device <- if (cuda_is_available()) torch_device("cuda") else torch_device("cpu")
cat("Using device:", as.character(device), "\n")
```

```
## Using device: cpu
```

Sampler from f(x;theta)

```
rinvgauss_vec <- function(n, mu, lambda) {
  nu <- rnorm(n); y <- nu^2
  x  <- mu + (mu^2 * y) / (2 * lambda) -
    (mu / (2 * lambda)) * sqrt(4 * mu * lambda * y + mu^2 * y^2)
  ifelse(runif(n) <= mu / (mu + x), x, mu^2 / x)
}

sim_one_dataset <- function(theta, k = 50) {
  if (theta < 1e-12) {
    z <- rnorm(k); 1 / (4 * z^2)              # Levy
  } else {
```

```
    rinvgauss_vec(k, mu = 1/(2*sqrt(theta)), lambda = 0.5)
  }
}

sample_prior <- function(K) {
  th <- numeric(K)
  for (i in 1:K) {
    u <- runif(1)
    if (u < 0.20)      th[i] <- 0
    else if (u < 0.70) th[i] <- rexp(1, 5)
    else               th[i] <- runif(1, 0, 2.5)
  }
  th
}
```

Batch generator with enriched features + expert stats

```
gen_batch <- function(B, k = 50) {
  thetas <- sample_prior(B)
  X <- matrix(NA, B, k); ES <- matrix(NA, B, 6)
  for (i in 1:B) {
    x <- sim_one_dataset(thetas[i], k); X[i, ] <- x
    ES[i, ] <- c(
      (1/(2*mean(x)))^2,        # MLE
      min(x), median(x),
      sd(log(x)), mean(1/x),
      quantile(x, 0.10, names = FALSE)
    )
  }
  # Enriched features [x, 1/x, log x] -> (B, k, 3)
  feat <- array(0, dim = c(B, k, 3))
  feat[,,1] <- X; feat[,,2] <- 1/X; feat[,,3] <- log(X)
  list(
    X     = torch_tensor(feat, dtype = torch_float()),
    S     = torch_tensor(ES,   dtype = torch_float()),
    theta = torch_tensor(matrix(thetas, ncol=1), dtype = torch_float())
  )
}
```

DeepSets with expert stats concatenation

```
deepset <- nn_module(
  initialize = function(d_in = 3, d_expert = 6, w = 128) {
    self$psi <- nn_sequential(
      nn_linear(d_in, w), nn_relu(),
      nn_linear(w, w),    nn_relu(),
      nn_linear(w, w),    nn_relu()
    )
    self$phi <- nn_sequential(
```

```
      nn_linear(w + d_expert, w), nn_relu(),
      nn_linear(w, w),            nn_relu(),
      nn_linear(w, w),            nn_relu(),
      nn_linear(w, 1),            nn_softplus()
    )
  },
  forward = function(X, S) {
    h <- self$psi(X)$mean(dim = 2)            # mean pool over k
    self$phi(torch_cat(list(h, S), dim = 2))
  }
)

weighted_loss <- function(pred, true) {
  w <- 1 / (true + 0.001)
  (w * (pred - true)^2)$mean()
}
```

Train one estimator

```
train_one <- function(seed, K = 20000, epochs = 100, batch = 256, k = 50) {
  torch_manual_seed(seed); set.seed(seed)
  model <- deepset()$to(device = device)
  opt   <- optim_adam(model$parameters, lr = 1e-3)

  val   <- gen_batch(2000, k)
  vX <- val$X$to(device=device); vS <- val$S$to(device=device); vT <- val$theta$to(device=device)

  for (e in 1:epochs) {
    model$train()
    for (s in 1:50) {                          # 50 steps per epoch
      b <- gen_batch(batch, k)
      bX <- b$X$to(device=device); bS <- b$S$to(device=device); bT <- b$theta$to(device=device)
      opt$zero_grad()
      loss <- weighted_loss(model(bX, bS), bT)
      loss$backward()
      opt$step()
    }
    if (e %% 10 == 0) {
      model$eval()
      with_no_grad({
        vl <- weighted_loss(model(vX, vS), vT)$item()
      })
      cat(sprintf("seed %d epoch %3d  val wMSE = %.4f\n", seed, e, vl))
    }
  }
  model
}
```

Train ensemble of 3

```
models <- list()
for (i in 1:3) {
  cat("\n=== Training member", i, "===\n")
  models[[i]] <- train_one(seed = 2026 + i*100)
}
```

```
##
## === Training member 1 ===
## seed 2126 epoch  10  val wMSE = 0.0705
## seed 2126 epoch  20  val wMSE = 0.0454
## seed 2126 epoch  30  val wMSE = 0.4755
## seed 2126 epoch  40  val wMSE = 0.4755
## seed 2126 epoch  50  val wMSE = 0.4755
## seed 2126 epoch  60  val wMSE = 0.4755
## seed 2126 epoch  70  val wMSE = 0.4755
## seed 2126 epoch  80  val wMSE = 0.4755
## seed 2126 epoch  90  val wMSE = 0.4755
## seed 2126 epoch 100  val wMSE = 0.4755
##
## === Training member 2 ===
## seed 2226 epoch  10  val wMSE = 0.0454
## seed 2226 epoch  20  val wMSE = 0.0447
## seed 2226 epoch  30  val wMSE = 0.0440
## seed 2226 epoch  40  val wMSE = 0.0589
## seed 2226 epoch  50  val wMSE = 0.0423
## seed 2226 epoch  60  val wMSE = 0.0424
## seed 2226 epoch  70  val wMSE = 0.0408
## seed 2226 epoch  80  val wMSE = 0.0485
## seed 2226 epoch  90  val wMSE = 0.0398
## seed 2226 epoch 100  val wMSE = 0.0399
##
## === Training member 3 ===
## seed 2326 epoch  10  val wMSE = 0.0506
## seed 2326 epoch  20  val wMSE = 0.0474
## seed 2326 epoch  30  val wMSE = 0.0455
## seed 2326 epoch  40  val wMSE = 0.0462
## seed 2326 epoch  50  val wMSE = 0.0437
## seed 2326 epoch  60  val wMSE = 0.0433
## seed 2326 epoch  70  val wMSE = 0.0435
## seed 2326 epoch  80  val wMSE = 0.0419
## seed 2326 epoch  90  val wMSE = 0.0431
## seed 2326 epoch 100  val wMSE = 0.0412
```

Apply ensemble to 1000 test datasets

```
# Build test tensors with enriched features + expert stats
B <- length(Z_test); k <- 50
Xfeat <- array(0, dim = c(B, k, 3)); ES <- matrix(NA, B, 6)
for (i in 1:B) {
```

```r
  x <- as.numeric(Z_test[[i]])
  Xfeat[i,,1] <- x; Xfeat[i,,2] <- 1/x; Xfeat[i,,3] <- log(x)
  ES[i, ] <- c((1/(2*mean(x)))^2, min(x), median(x),
               sd(log(x)), mean(1/x), quantile(x, 0.10, names=FALSE))
}
tX <- torch_tensor(Xfeat, dtype=torch_float())$to(device=device)
tS <- torch_tensor(ES,    dtype=torch_float())$to(device=device)

preds <- matrix(NA, 3, B)
for (i in 1:3) {
  models[[i]]$eval()
  with_no_grad({ preds[i, ] <- as.numeric(models[[i]](tX, tS)$cpu()) })
}
```

save

```r
# Drop the dead member 1, average only 2 and 3
theta_nbe <- pmax(colMeans(preds[2:3, ]), 0)
summary(theta_nbe)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.0000000 0.0002443 0.0102526 0.3198716 0.3615355 2.1990130
```

```r
# Save
save(theta_mle, theta_nbe, file = "HW3_P2_estimates.RData")
write.csv(data.frame(MLE=theta_mle, NBE=theta_nbe), "HW3_P2.csv", row.names=FALSE)
```

part 3 mcmc

```r
library(Rcpp)

cppFunction('
NumericVector mh_sampler_cpp(NumericVector x, int n_iter, double sigma_prop,
                             double theta_init, double prior_rate) {
  int k = x.size();
  double sum_x = 0.0, sum_log_x = 0.0, sum_inv_x = 0.0;
  for (int i = 0; i < k; i++) {
    sum_x     += x[i];
    sum_log_x += log(x[i]);
    sum_inv_x += 1.0 / x[i];
  }
  double C = -0.5 * k * log(4.0 * M_PI) - 1.5 * sum_log_x - 0.25 * sum_inv_x;

  NumericVector chain(n_iter);
  chain[0] = theta_init;

  double log_post_cur = C - theta_init * sum_x + k * sqrt(theta_init)
                        + log(prior_rate) - prior_rate * theta_init;
```

```
  for (int t = 1; t < n_iter; t++) {
    double theta_cur = chain[t - 1];
    double log_theta_star = log(theta_cur) + R::rnorm(0.0, sigma_prop);
    double theta_star = exp(log_theta_star);

    double log_post_star = C - theta_star * sum_x + k * sqrt(theta_star)
                             + log(prior_rate) - prior_rate * theta_star;

    // Jacobian for log-transform proposal
    double log_alpha = log_post_star - log_post_cur +
                       log(theta_star) - log(theta_cur);

    if (log(R::runif(0.0, 1.0)) < log_alpha) {
      chain[t]    = theta_star;
      log_post_cur = log_post_star;
    } else {
      chain[t] = theta_cur;
    }
  }
  return chain;
}
')

# R wrapper
mcmc_sample <- function(x, n_iter = 20000, burn = 5000,
                        theta_init = NULL, sigma_prop = 0.3,
                        prior_rate = 3) {
  if (is.null(theta_init)) theta_init <- max((1/(2*mean(x)))^2, 1e-6)
  chain <- mh_sampler_cpp(x, n_iter, sigma_prop, theta_init, prior_rate)
  list(chain = chain, burn = burn,
       post = chain[(burn + 1):n_iter])
}
```

```
res1 <- mcmc_sample(Z_test[[1]], n_iter = 30000, burn = 5000, sigma_prop = 0.3)
chain_post <- res1$post

cat("Posterior mean:  ", round(mean(chain_post), 5), "\n")
```

```
## Posterior mean:   0.01133
```

```
cat("Posterior median:", round(median(chain_post), 5), "\n")
```

```
## Posterior median: 0.00979
```

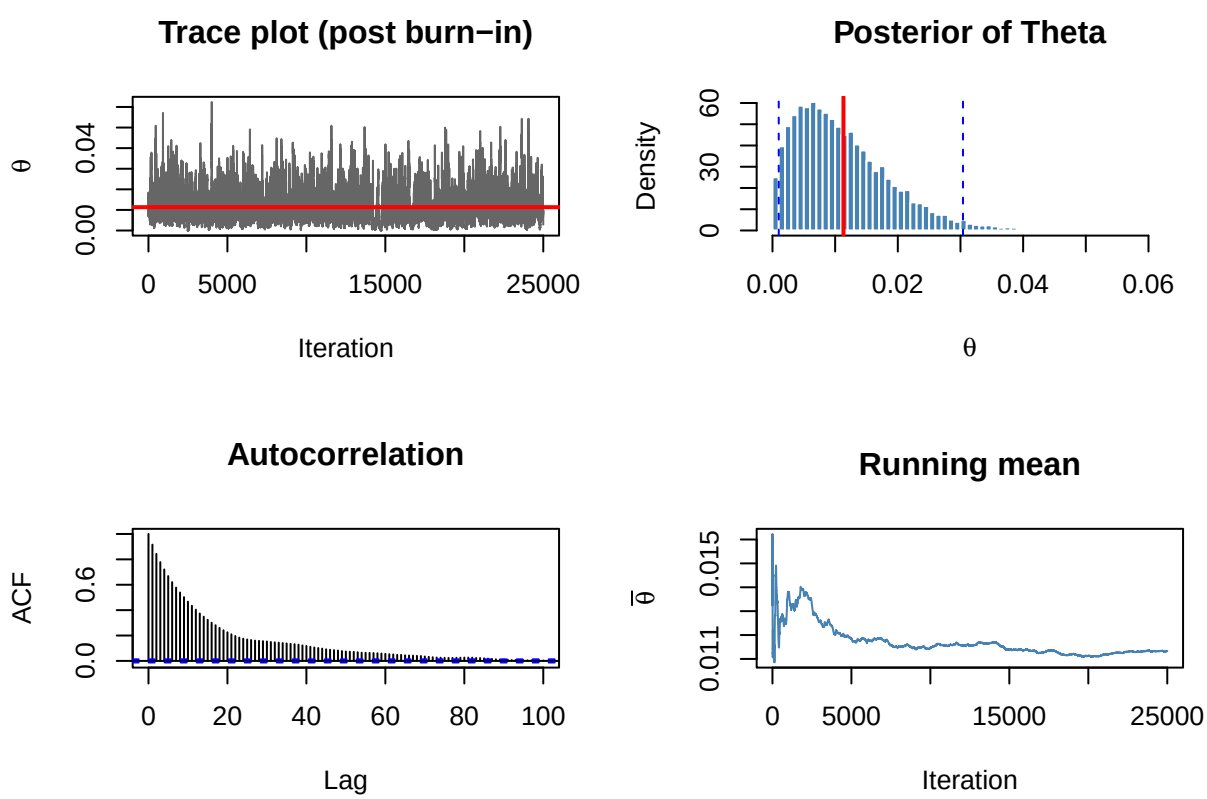
```
cat("95% CI:          [",
    round(quantile(chain_post, 0.025), 5), ",",
    round(quantile(chain_post, 0.975), 5), "]\n")
```

```
## 95% CI:          [ 0.001 , 0.0304 ]
```

```r
cat("ESS:              ", round(coda::effectiveSize(chain_post), 1), "\n")
```

```
## ESS:              799.9
```

```r
par(mfrow = c(2, 2))
plot(chain_post, type = "l", col = "grey40",
     main = "Trace plot (post burn-in)",
     xlab = "Iteration", ylab = expression(theta))
abline(h = mean(chain_post), col = "red", lwd = 2)

hist(chain_post, breaks = 50, col = "steelblue", border = "white",
     probability = TRUE, main = "Posterior of Theta", xlab = expression(theta))
abline(v = mean(chain_post), col = "red", lwd = 2)
abline(v = quantile(chain_post, c(0.025, 0.975)), col = "blue", lty = 2)

acf(chain_post, lag.max = 100, main = "Autocorrelation")

rm_vals <- cumsum(chain_post) / seq_along(chain_post)
plot(rm_vals, type = "l", col = "steelblue",
     main = "Running mean", xlab = "Iteration", ylab = expression(bar(theta)))
```




```r
par(mfrow = c(1, 1))
```

```r
theta_mcmc <- numeric(length(Z_test))

t0 <- Sys.time()
for (i in seq_along(Z_test)) {
  res <- mcmc_sample(Z_test[[i]], n_iter = 20000, burn = 5000, sigma_prop = 0.3)
  theta_mcmc[i] <- mean(res$post)
  if (i %% 100 == 0) {
    cat("Done", i, "/", length(Z_test),
        " elapsed:", round(as.numeric(Sys.time() - t0, units = "secs"), 1), "s\n")
  }
}
```

```
## Done 100 / 1000  elapsed: 0.4 s
## Done 200 / 1000  elapsed: 0.8 s
## Done 300 / 1000  elapsed: 1.1 s
## Done 400 / 1000  elapsed: 1.5 s
## Done 500 / 1000  elapsed: 1.9 s
## Done 600 / 1000  elapsed: 2.3 s
## Done 700 / 1000  elapsed: 2.6 s
## Done 800 / 1000  elapsed: 3 s
## Done 900 / 1000  elapsed: 3.4 s
## Done 1000 / 1000  elapsed: 3.7 s
```

```r
cat("\nSummary of MCMC posterior means:\n")
```

```
##
## Summary of MCMC posterior means:
```

```r
summary(theta_mcmc)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.000000 0.008649 0.022991 0.292423 0.418802 2.275749
```

```r
save(theta_mle, theta_nbe, theta_mcmc, file = "HW3_P2_estimates.RData")
write.csv(data.frame(MLE = theta_mle, NBE = theta_nbe, MCMC = theta_mcmc),
          "HW3_P2.csv", row.names = FALSE)
```

part4

```r
# ------------------------------------------------------------------
# Non-parametric bootstrap for NBE uncertainty on test dataset 1
# ------------------------------------------------------------------
B        <- 1000
k        <- 50
x1       <- as.numeric(Z_test[[1]])

# Build bootstrap sample tensor: B datasets, each of size k,
# each resampled with replacement from x1
```

```r
set.seed(2026)
boot_idx <- matrix(sample(1:k, B * k, replace = TRUE), nrow = B, ncol = k)

# Construct enriched features and expert stats for each bootstrap sample
Xboot <- array(0, dim = c(B, k, 3))
Sboot <- matrix(NA, B, 6)
for (b in 1:B) {
  xb <- x1[boot_idx[b, ]]
  Xboot[b, , 1] <- xb
  Xboot[b, , 2] <- 1 / xb
  Xboot[b, , 3] <- log(xb)
  Sboot[b, ] <- c(
    (1 / (2 * mean(xb)))^2,
    min(xb), median(xb),
    sd(log(xb)), mean(1 / xb),
    quantile(xb, 0.10, names = FALSE)
  )
}

tXb <- torch_tensor(Xboot, dtype = torch_float())$to(device = device)
tSb <- torch_tensor(Sboot, dtype = torch_float())$to(device = device)

# Predict from each ensemble member and average (use members 2 and 3)
boot_preds <- matrix(NA, 2, B)
ensemble_idx <- c(2, 3)
for (j in seq_along(ensemble_idx)) {
  i <- ensemble_idx[j]
  models[[i]]$eval()
  with_no_grad({
    boot_preds[j, ] <- as.numeric(models[[i]](tXb, tSb)$cpu())
  })
}
theta_boot <- pmax(colMeans(boot_preds), 0)

# Point estimate for dataset 1 (original, not bootstrapped)
theta_hat_1 <- theta_nbe[1]

# Summary
cat("NBE bootstrap results for test dataset 1:\n")
```

```
## NBE bootstrap results for test dataset 1:
```

```r
cat("  Point estimate:   ", round(theta_hat_1, 5), "\n")
```

```
##   Point estimate:    0.00074
```

```r
cat("  Bootstrap mean:   ", round(mean(theta_boot), 5), "\n")
```

```
##   Bootstrap mean:    0.00587
```

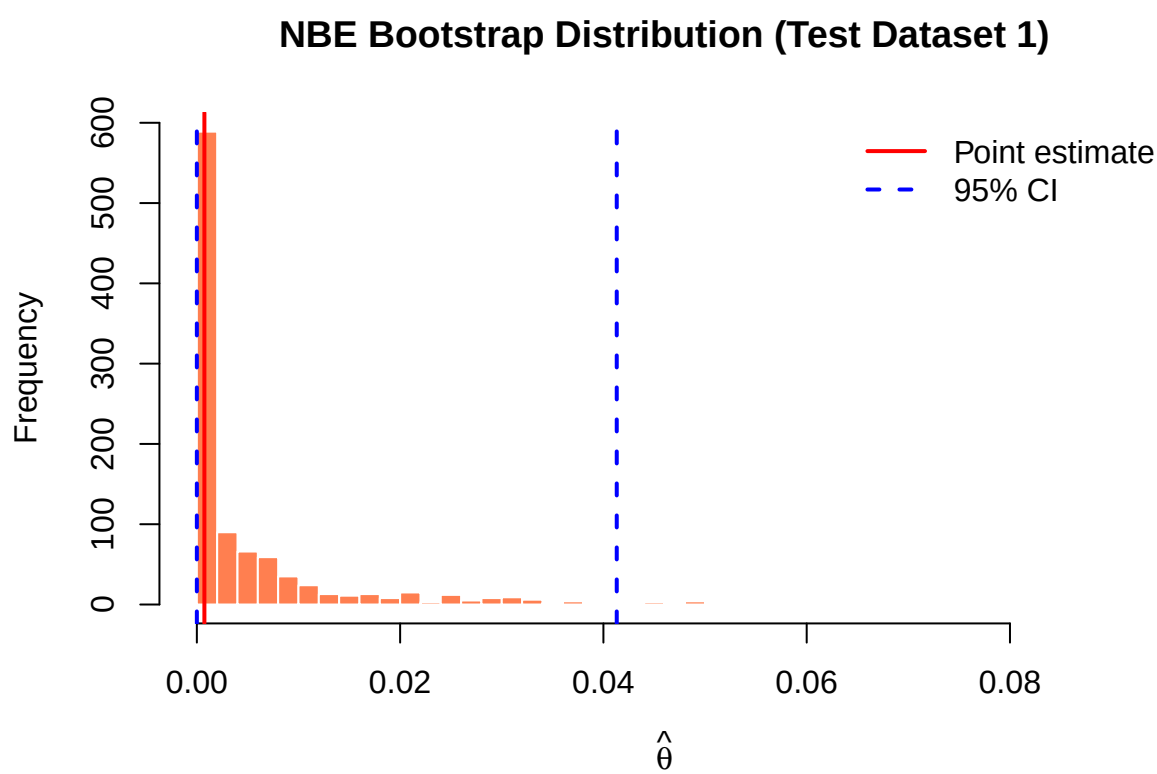
```r
cat("  Bootstrap SD:     ", round(sd(theta_boot), 5), "\n")
```

```
##   Bootstrap SD:      0.01149
```

```r
cat("  95% bootstrap CI: [",
    round(quantile(theta_boot, 0.025), 5), ",",
    round(quantile(theta_boot, 0.975), 5), "]\n")
```

```
##   95% bootstrap CI: [ 0 , 0.04131 ]
```

```r
# Histogram
hist(theta_boot, breaks = 40, col = "coral", border = "white",
     main = "NBE Bootstrap Distribution (Test Dataset 1)",
     xlab = expression(hat(theta)))
abline(v = theta_hat_1, col = "red", lwd = 2)
abline(v = quantile(theta_boot, c(0.025, 0.975)), col = "blue", lty = 2, lwd = 2)
legend("topright",
       legend = c("Point estimate", "95% CI"),
       col = c("red", "blue"), lty = c(1, 2), lwd = 2, bty = "n")
```

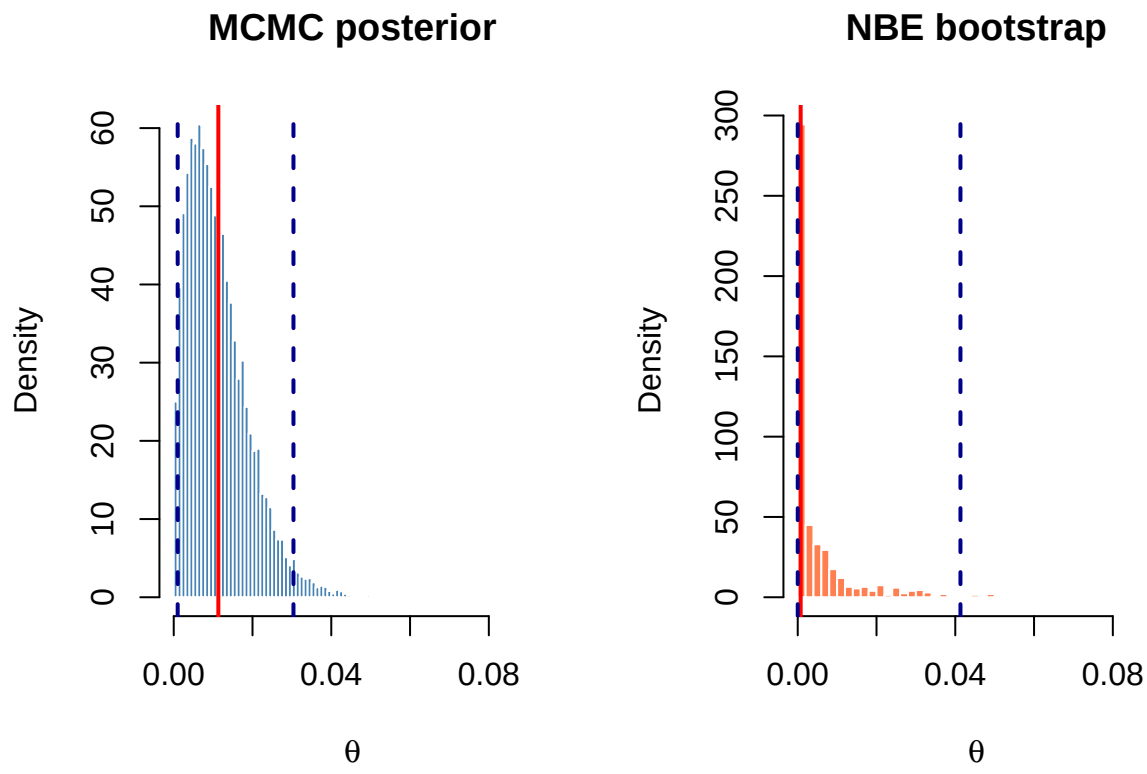



```r
# MCMC results (from earlier chunk)
mcmc_mean <- mean(chain_post)
mcmc_ci   <- quantile(chain_post, c(0.025, 0.975))
```

```r
mcmc_sd   <- sd(chain_post)

# NBE bootstrap results
nbe_mean  <- mean(theta_boot)
nbe_ci    <- quantile(theta_boot, c(0.025, 0.975))
nbe_sd    <- sd(theta_boot)

comparison <- data.frame(
  Method     = c("MCMC", "NBE bootstrap"),
  PointEst   = c(round(mcmc_mean, 5), round(theta_hat_1, 5)),
  SD         = c(round(mcmc_sd, 5),   round(nbe_sd, 5)),
  CI_lower   = c(round(mcmc_ci[1], 5), round(nbe_ci[1], 5)),
  CI_upper   = c(round(mcmc_ci[2], 5), round(nbe_ci[2], 5)),
  CI_width   = c(round(diff(mcmc_ci), 5), round(diff(nbe_ci), 5))
)

print(comparison, row.names = FALSE)
```

```
##         Method PointEst      SD CI_lower CI_upper CI_width
##           MCMC  0.01133 0.00780    0.001  0.03040  0.02940
##  NBE bootstrap  0.00074 0.01149    0.000  0.04131  0.04131
```

```r
# Overlay plot
par(mfrow = c(1, 2))
hist(chain_post, breaks = 50, col = "steelblue", border = "white",
     probability = TRUE,
     main = "MCMC posterior", xlab = expression(theta),
     xlim = range(c(chain_post, theta_boot)))
abline(v = mcmc_mean, col = "red", lwd = 2)
abline(v = mcmc_ci,  col = "darkblue", lty = 2, lwd = 2)

hist(theta_boot, breaks = 50, col = "coral", border = "white",
     probability = TRUE,
     main = "NBE bootstrap", xlab = expression(theta),
     xlim = range(c(chain_post, theta_boot)))
abline(v = theta_hat_1, col = "red", lwd = 2)
abline(v = nbe_ci, col = "darkblue", lty = 2, lwd = 2)
```




```r
par(mfrow = c(1, 1))
```